

SSD 2026 - Project Report S4

Group symbol: 2

Team: 2

Project title: Car Rental Management System

Team members

No	Name	Surname	Student ID	Role
1	Leon Jakub	Wyrobek Smeder	295238	PM, Team Leader
2	Wojciech	Wojnarowski	263594	Team member
3	Aleksandra	Król	272530	Team member
4	Youssef	Ayadi	294635	Team member

4. Construction and tests (S4)

4.1. Implementation

This section describes how the Car Rental Management System was built and what was delivered as the minimum viable product (MVP).

System description

The system is a web application for managing car rentals. Customers can browse vehicles, make reservations, pay for bookings, and view their rental history. Employees can manage the fleet, process vehicle pickup and return, view customers, and generate basic reports.

The application follows the architecture defined in S3:

- **Frontend** — user interface for customers and employees (React)
- **Backend** — business logic and REST API (Spring Boot)
- **Database** — persistent storage of users, vehicles, reservations, and payments (H2 for development, PostgreSQL for deployment)

Payment processing and GPS tracking are **simulated** in the MVP, as described in the project specification. The payment flow mimics an external provider; GPS tracking is activated automatically when an employee processes a vehicle pickup.

Backend source code (backend/src/main/java/com/carrental/backend/)

Package/module	Artefacts	Type	Responsibility
BackendApplication.java	1 class	Entry point	Starts the application
user/	User, roles, repositories, services, controllers, DTOs	Domain + API	Registration, login, logout, user management
vehicle/	Vehicle, repository, service, controller	Domain + API	Fleet catalog, filtering, staff CRUD
reservation/	Reservation, status, repository, service, controller, request DTOs	Domain + API	Booking, lifecycle, agreement download
payment/	Payment, status, repository, service, controller	Domain + API	Simulated payment create / confirm / fail
report/	Service, controller, response DTO	Domain + API	Revenue and fleet utilization reports
security/	Config, token service, filter, utilities	Cross cutting	Authentication, authorization
exception/	Global exception handler	Cross cutting	Consistent error responses

Frontend source code (frontend/src/)

Directory	Artefacts	Type	Responsibility
pages/	Home, Auth, Reservation, Dashboard, StaffPanel, Privacy	UI pages	Customer and employee workflows
components/	common/, layout/, vehicles/	UI components	Reusable UI, navbar, vehicle cards, filters
services/	api, user, vehicle, reservation, payment, report services	API client	Communication with the backend
hooks/	useUser, useVehicles, useReservations	React hooks	Data fetching
store/	appStore	State	Logged-in user, notifications
types/	User, Vehicle, Reservation, Payment	TypeScript types	Data models

utils/	dateUtils, formatters, constants	Utilities	Formatting and helpers
--------	----------------------------------	-----------	------------------------

Configuration files

File	Type	Purpose
backend/pom.xml	Maven	Backend dependencies and build
backend/src/main/resources/application.example.properties	Properties	Local configuration template
backend/src/main/resources/application.h2.properties	Properties	H2 database profile
backend/src/main/resources/application.postgres.properties	Properties	PostgreSQL profile
backend/src/test/resources/application.properties	Properties	Test database configuration
frontend/package.json	npm	Frontend dependencies and scripts
frontend/vite.config.ts	Vite	Dev server and API proxy
frontend/tailwind.config.js	CSS	Styling configuration
frontend/.env.example	Environment	Optional API URL override

Database migrations (backend/src/main/resources/db/migration/)

Migration	Content
V1__init.sql	Core tables: users, vehicles, reservations, payments
V2__seed.sql	Demo vehicles and users
V3__user_passwords.sql	Password column
V4__reservation_details.sql	Contact, licence, insurance fields
V5__reservation_return_notes.sql	Return notes
V6__user_roles.sql	Customer / employee / admin roles
V7__demo_user_passwords.sql	Demo account passwords
V8__return_inspection_fields.sql	Damage notes and extra charges
V9__demo_completed_reservations.sql	Sample data for reports

Program resources

Resource	Location	Purpose
Vehicle images	frontend/public/vehicles/	Catalog photos
Hero image	frontend/src/assets/hero.png	Landing page

Resource	Location	Purpose
HTTP request samples	backend/src/main/resources/http_requests/	Manual API testing
README	project root	Setup and demo instructions

Demo accounts (password: password):

Email	Role
john.smith@example.com	CUSTOMER
employee@rentacar.com	EMPLOYEE

Relationships between artefacts

The frontend communicates with the backend through HTTP requests. The backend processes business rules and reads or writes data in the database. Security rules ensure that customers and employees only access the features allowed for their role.

MVP coverage

The implementation covers the main functional requirements from S2:

- Vehicle browsing, filtering, and details
- Reservation creation with price calculation
- User registration, login, and logout
- Simulated payment and rental agreement generation
- Customer reservation history and cancellation
- Employee pickup, return, and fleet management
- Basic reports (revenue, reservations, fleet utilization)

Some requirements are only partially met or simulated:

- Vehicle **sorting** is not available in the UI (filtering works)
- **Payment** and **GPS** use mock integrations instead of real external services
- **Customer management** for employees is read-only

Application execution (demo)

A live demonstration of the running application is part of the project evaluation. The demo follows this scenario:

Customer flow

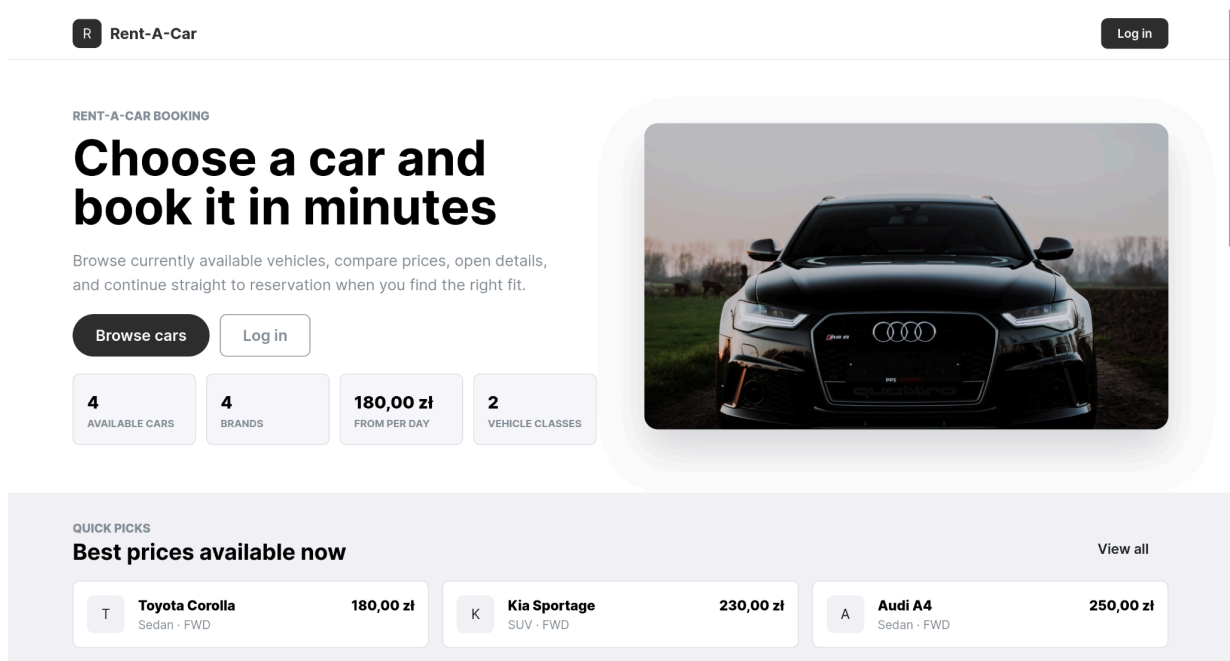
1. Log in as a customer
2. Browse and filter vehicles
3. Create a reservation (dates, personal data, optional insurance)
4. Confirm simulated payment
5. View the confirmed booking and download the rental agreement

Employee flow

1. Log in as an employee
2. Process vehicle pickup for a confirmed reservation
3. Process vehicle return (optionally record damage or extra charges)
4. Generate a report for a selected date range

Application screenshots

Figure 1 — Home page / vehicle catalog



R

Rent-A-Car

Log in

Find your ride

Showing 4 of 4 available vehicles

Search make or model...

Type

☐ SUV

☐ Sedan

Make

☐ Audi

☐ BMW

☐ Kia

☐ Toyota

Model

☐ A4

☐ Corolla

☐ Sportage

☐ X5

Drive

☐ AWD

☐ FWD

Sort by

Price: low to high

Max price / day

180,00 ZL/DAY

No Image

SEDAN • FWD

Toyota Corolla

details

reserve

230,00 ZL/DAY

No Image

SUV • FWD

Kia Sportage

details

reserve

250,00 ZL/DAY

No Image

SEDAN • FWD

Audi A4

details

reserve

420,00 ZL/DAY

No Image

SUV • AWD

BMW X5

details

reserve

Figure 2 — Reservation and payment

R

Rent-A-Car

J John

Reserve

Toyota Corolla • 180,00 zł / day

Personal details

Full name

John Smith

Email

john.smith@example.com

Telephone

6143626124

Driving licence ID

2987349823F

Pick-up date

24 / 06 / 2026

Return date

27 / 06 / 2026

Insurance & fees

Insurance

☐ None

☒ Regular (+15,00 Zł/Day)

☐ Premium (+30,00 Zł/Day)

☐ Young driver fee (+25,00 zł)

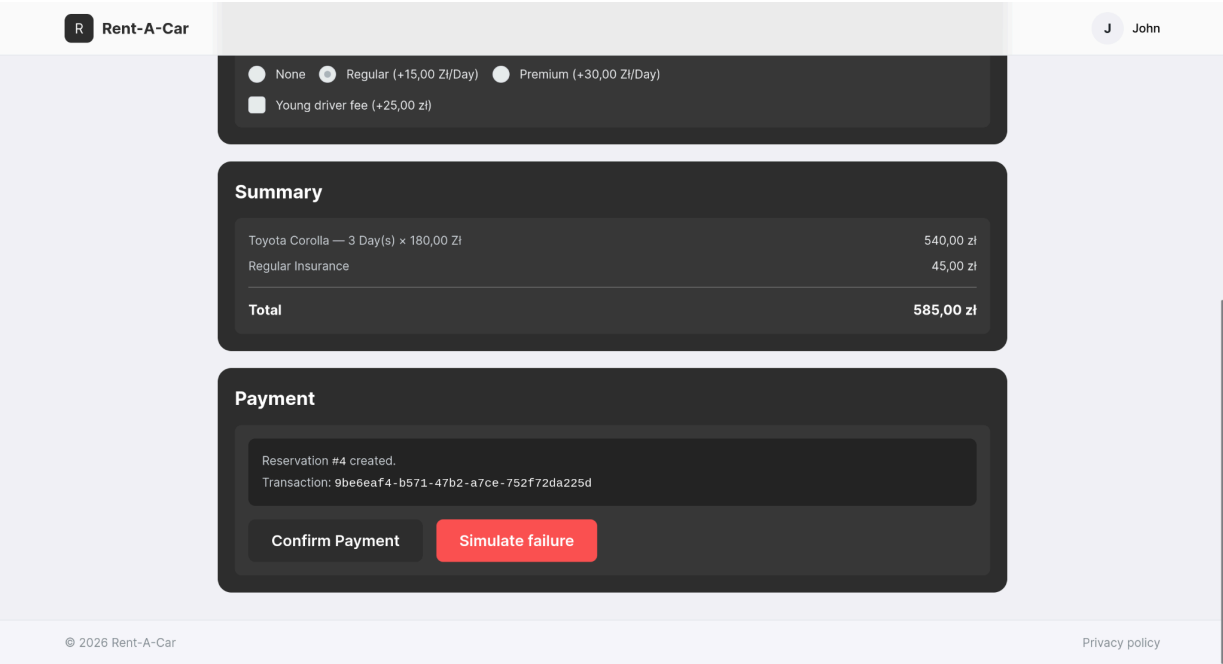


Figure 3 — Customer dashboard

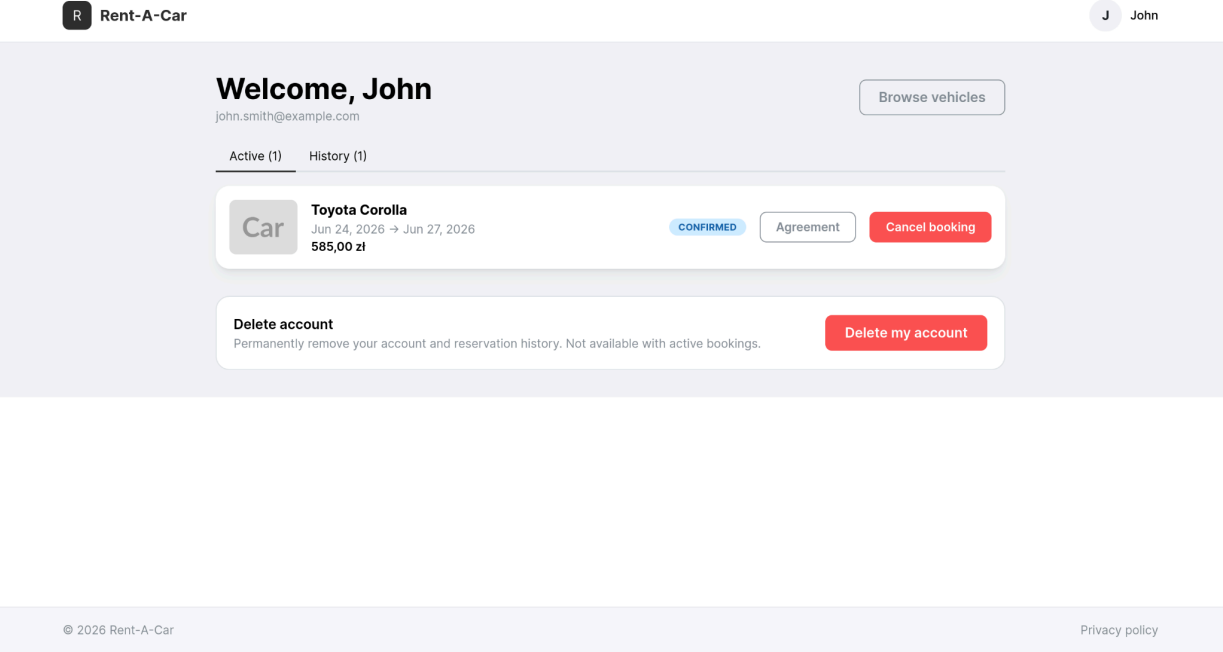


Figure 4 — Staff panel (pickup / return)

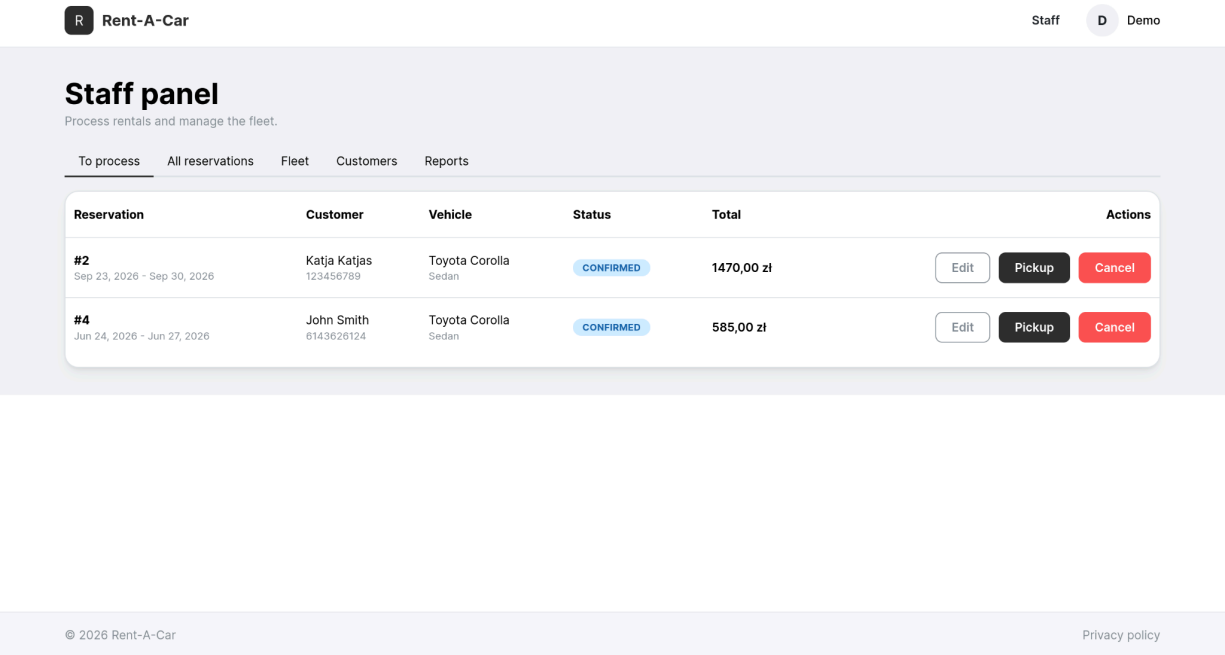
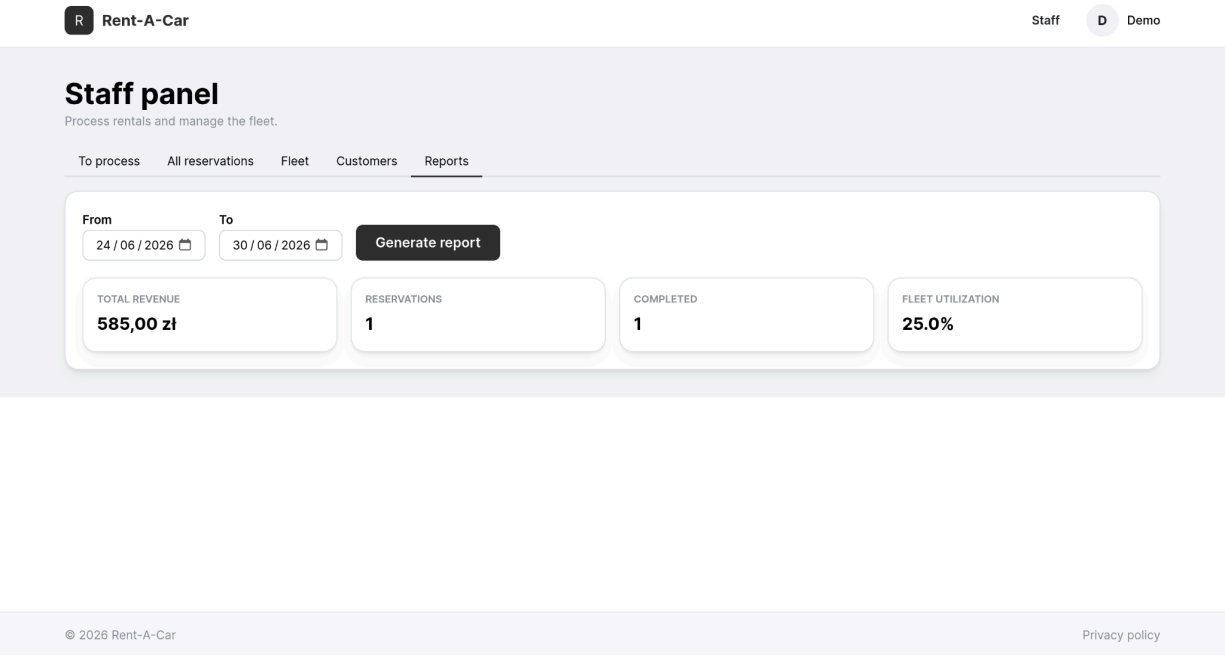


Figure 5 — Reports



4.2. Tests

Testing was carried out to verify that the system meets the requirements defined in S2. Tests cover both **functional** behaviour (business processes, user interface, data exchange) and **non-functional** aspects (performance, usability, security).

Tests are divided into:

- **Automated tests** — run on the backend during development
- **Manual tests** — performed through the web interface during demo preparation

All automated backend tests passed (8 tests, 0 failures). The frontend builds successfully.

4.2.1. Requirements tests

4.2.1.1. Functional requirements tests

Automated tests

Requirement area	What was tested	Result
FR-09 Reservation creation	Creating a reservation for an available vehicle	Pass
FR-09 Double booking prevention	Rejecting a reservation that overlaps an existing booking	Pass
FR-03 Payment	Payment creation and confirmation update the reservation status	Pass
FR-12 Authentication	Login returns a valid session token	Pass
FR-12 Authorization	Protected pages reject unauthenticated access	Pass
FR-05 Reports	Employees can access reports; customers cannot	Pass

Manual tests

Requirement	Test scenario	Result
FR-01	Filter vehicles by type and price	Pass
FR-02	Price breakdown shown before booking confirmation	Pass
FR-07	Browse vehicle catalog on the home page	Pass
FR-08	View detailed vehicle information	Pass
FR-10	Enter personal data during reservation	Pass
FR-11	Register a new customer account	Pass
FR-13	Cancel a reservation from the customer dashboard	Pass
FR-14	Download rental agreement after payment	Pass

Requirement	Test scenario	Result
FR-15	Employee processes vehicle pickup	Pass
FR-16	Employee processes vehicle return with inspection notes	Pass
FR-17	Employee adds, edits, and removes vehicles	Pass
FR-20	Customer views rental history	Pass

Example manual test — complete booking

Step	Action	Expected result
1	Customer logs in	Home page with vehicle catalog
2	Selects a vehicle and opens reservation form	Form with dates and personal data
3	Chooses rental dates and insurance	Total price updates
4	Submits reservation and confirms payment	Reservation status: Confirmed
5	Opens dashboard and downloads agreement	Agreement file downloaded

4.2.1.2. Non-functional requirements tests

Requirement	Verification method	Scenario	Result
NF-01 Booking under 5 minutes	Manual timing during demo rehearsal	Full customer booking flow	Pass (~3 minutes)
NF-02 Response time	Manual observation	Loading vehicle list and pages	Pass (under 2 seconds locally)
NF-03 Concurrent users	Manual test	Customer and employee using the system at the same time	Pass
NF-04 Browser compatibility	Manual test	Chrome and Firefox	Pass
NF-05 Simple reservation process	Usability review	Steps from vehicle selection to confirmation	Pass
NF-06 Deployment	Manual test	Backend and frontend run from built artefacts	Pass
NF-07 Scalability	Design review	Stateless API and database backed storage	Acceptable for MVP
NF-08 GDPR	Review	Privacy page, limited data collection, no card storage	Pass (basic compliance)
NF-09 Secure data storage	Automated + manual	Password hashing, role-based access, protected endpoints	Pass

4.2.2. Remaining tests

Integration tests (automated)

- Application starts correctly
- Authentication flow

- Reservation and payment logic
- Report access control

Manual regression checks

- Logged-out users cannot access customer or staff pages
- Customers cannot access the staff panel
- Invalid login shows an error message
- Conflicting reservation dates are rejected